[2CEIT401 – Operating Systems]

Practical: 2

AIM- Introduction of basic shell programming. Write shell script using operators.

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥
U.V. Patel College of Engineering

Department of Computer Engineering

Aim: Introduction of basic shell programming. Write shell script using operators.

1. Explain basic command of Vi Editor

Ans:

Vi (Visual Editor) is a text editor in Unix/Linux used to create and modify files.
Vi works in three main modes:

Command Mode (Default Mode)

- Used to navigate, delete, copy, paste, and execute commands.
- You enter this mode by pressing Esc.

Basic Navigation Commands

Command	Function
h	Move cursor left
l	Move cursor right
j	Move cursor down
k	Move cursor up
w	Move to next word
b	Move to previous word
0	Move to beginning of line
$	Move to end of line
gg	Go to first line
G	Go to last line

Insert Mode

- Used to insert or modify text.
- Entered from command mode.

Command	Description
i	Insert before cursor
I	Insert at beginning of line
a	Append after cursor
A	Append at end of line
o	Open new line below
O	Open new line above

Press `Esc` to return to command mode.

Last Line (Ex) Mode

- Used for saving, quitting, searching, etc.
- Entered by pressing : in command mode.

Command	Function
:w	Save file
:q	Quit
:wq	Save and quit
:q!	Quit without saving
:set number	Show line numbers

2. Editing Command of Vi Editor.

Ans:

Editing commands are mainly used in Command Mode.

Delete Commands

Command	Action
x	Delete character under cursor
dd	Delete entire line
dw	Delete a word
D	Delete from cursor to end of line
d$	Delete till end of line

Copy (Yank) and Paste Commands

Command	Action
yy	Copy (yank) a line
yw	Copy a word
p	Paste after cursor
P	Paste before cursor

Replace and Change Commands

Command	Action
r	Replace single character
R	Replace continuously
cw	Change a word
cc	Change entire line

Undo and Redo

Command	Function
u	Undo last action
U	Undo entire line
Ctrl + r	Redo

3. Explain I/O redirection, Piping and Command Substitution

Ans:

I/O Redirection

I/O redirection allows changing the standard input/output of commands.

- Standard Streams
- stdin (0) → keyboard
- stdout (1) → screen
- stderr (2) → error messages

Types of Redirection

1. Output Redirection

   ls > file.txt

   Writes output to file.txt (overwrites).

   ls >> file.txt

   Appends output to file.

2. Input Redirection

   sort < data.txt

   Takes input from data.txt.

3. Error Redirection

   ls xyz 2> error.txt

Redirects error to file.

4. Both Output and Error

command > out.txt 2>&1

Piping (|)

Piping sends the output of one command as input to another.

Syntax

command1 | command2

Examples

ls | wc -l

```
[24012011142@linuxserv ~]$ ls | wc -l
3
```

Counts number of files.

ps aux | grep root

Searches processes related to root.

Command Substitution

Command substitution allows using output of a command as an argument.

Syntax

`command`

or

$(command)

Example

date=$(date)

echo "Today is $date"

```
[24012011142@linuxserv ~]$ date=$(date)
[24012011142@linuxserv ~]$ echo "Today is $date"
Today is Wed Jan 28 15:08:58 IST 2026
[24012011142@linuxserv ~]$
```

&

files=$(ls | wc -l)

echo "Total files: $files"

```
[24012011142@linuxserv ~]$ files=$(ls | wc -l)
[24012011142@linuxserv ~]$ echo "Total files: $files"
Total files: 3
[24012011142@linuxserv ~]$
```

4. What is shell script? Explain with Examples.

Ans:

Definition

A shell script is a text file containing a sequence of shell commands that are executed automatically by the shell.

Shell scripts are used for:

- Automation
- System administration
- File handling
- Task scheduling

Structure of a Shell Script

#!/bin/bash

 # This is a comment

echo "Hello World"

- #!/bin/bash → Shebang (interpreter path)
- echo → Prints output

Example 1: Simple Shell Script

echo "Welcome to Shell Programming"

```
[24012011142@linuxserv prac2]$ sh que_4.1.sh
Welcome To Shell Programming
```

Example 2: Using Variables

name="Vatsal"

echo "Hello $name"

```
[24012011142@linuxserv prac2]$ sh que_4.2.sh
Hello Vatsal
```

Example 3: Arithmetic Operators

a=10

b=5

sum=$((a + b))

echo "Sum = $sum"

```
[24012011142@linuxserv prac2]$ sh que_4.3.sh
Sum = 15
```

Example 4: Conditional Statement

```
#!/bin/bash

read -p "Enter number: " n

if [ $n -gt 0 ]

then

  echo "Positive number"

else

  echo "Negative number"

fi
```

```
[24012011142@linuxserv prac2]$ sh que_4.4.sh
Enter Number: 5
Positive Number
[24012011142@linuxserv prac2]$ sh que_4.4.sh
Enter Number: -5
Negative Number
```

Advantages of Shell Script

- Saves time
- Reduces manual work
- Easy to write and modify
- Powerful automation too

5. Write a shell script to scan your name and display it like My college Name is: UVPCE

Code:

```
echo "Enter Your College Name: "

read college_name

echo "My College Name is: $college_name"
```

Output:

```
[24012011142@linuxserv prac2]$ sh que_5.sh
Enter Your College Name:
UVPCE
My College name is: UVPCE
```

6. Write a shell script to scan two variables and to display their sum, mul, div, sub and modulo division. Write a shell script to scan two variables and to display their sum, mul, div, sub and modulo division as per the user choice. (no need to continue, only once is OK)

Code1:

```
echo "Enter Two Numbers: "

read num1

read num2

sum=`expr $num1 + $num2`

sub=`expr $num1 - $num2`

mul=`expr $num1 \* $num2`

div=`expr $num1 / $num2`

mod=`expr $num1 % $num2`

echo "Sum: $sum"

echo "Sub: $sub"

echo "Mul: $mul"

echo "Div: $div"

echo "Mod: $mod"
```

Output1:

```
[24012011142@linuxserv prac2]$ sh que_6.1.sh
Enter Two Numbers
15
16
Sum: 31
Sub: -1
Mul: 240
Div: 0
Mod: 15
```

Code2:

```
echo "Enter Two Numbers: "
read num1
read num2
echo "Enter Choice "
echo "1. Add"
echo "2. Sub"
echo "3. Mul"
echo "4. Div"
echo "5. Mod"
read choice
if [ $choice = 1 ] then
    sum=`expr $num1 + $num2`
    echo "Sum: $sum"
elif [ $choice = 2 ]; then
    sub=`expr $num1 - $num2`
    echo "Subtraction: $sub"
elif [ $choice = 3 ]; then
    mul=`expr $num1 \* $num2`
    echo "Multiplication: $mul"
elif [ $choice = 4 ]; then
    div=`expr $num1 / $num2`
    echo "Division: $div"
elif [ $choice = 5 ]; then
    mod=`expr $num1 % $num2`
```

```
    echo "Modulus: $mod"
fi
```

Output2:

```
[24012011142@linuxserv prac2]$ sh que_6.2.sh
Enter Two Numbers
15
16
Enter Choise
1. Sum
2. Sub
3. Mul
4. div
5. mod
3
Mul: 240
```

7. Write a shell script to find greatest of two. Script must consider the case where two numbers are equal.

Code:

```
echo "Enter Two Numbers: "
read num1
read num2
if [ $num1 -eq $num2 ]; then
    echo "Both are Equal"
elif [ $num1 -gt $num2 ]; then
    echo "$num1 is Greater than $num2"
else
    echo "$num2 is Greater than $num1"
fi
```

Output:

```
[24012011142@linuxserv prac2]$ sh que_7.sh
Enter Two Numbers
15
12
15 is Greater then 12
```

8. Write a shell script to find greatest of three. Script must consider the case where two numbers are equal.

Code:

```
echo "Enter Number1:"

read a

echo "Enter Number2:"

read b

echo "Enter Number3:"

read c


if [ $a -gt $b ] && [ $a -gt $c ]

then

   echo "$a is greatest"

elif [ $b -gt $a ] && [ $b -gt $c ]

then

   echo "$b is greatest"

elif [ $c -gt $a ] && [ $c -gt $b ]

then

   echo "$c is greatest"


elif [ $a -eq $b ] && [ $a -gt $c ]
```

```
then
    echo "$a and $b are equal and greatest"
elif [ $a -eq $c ] && [ $a -gt $b ]
then
    echo "$a and $c are equal and greatest"
elif [ $b -eq $c ] && [ $b -gt $a ]
then
    echo "$b and $c are equal and greatest"


elif [ $a -eq $b ] && [ $b -eq $c ]
then
    echo "All three numbers are equal"
fi
```

Output:

```
[24012011142@linuxserv prac2]$ sh que_8.sh
Enter Number1:
12
Enter Number2:
12
Enter Number3:
10
12 and 12 are equal and greatest
[24012011142@linuxserv prac2]$
```